

MongoDB: Updating and Replacing Documents

1. Introduction

MongoDB provides multiple operations to manipulate documents within collections. In this tutorial, two key operations are covered:

1. **Updating documents**
2. **Replacing documents**

Operations discussed:

- `updateOne()`
- `updateMany()`
- Update operators: `$set`, `$unset`, `$rename`, `$inc`, `$mul`, `$currentDate`, `$min`, `$max`
- Array update operators: `$push`, `$addToSet`, `$pop`, `$pull`, `$pullAll`
- `replaceOne()`

Examples are demonstrated using a **students** collection.

2. Updating Documents in MongoDB

MongoDB provides two main update commands:

- `updateOne()` – updates only the first matched document.
- `updateMany()` – updates all matched documents.

Both commands use the same structure:

2.1 Syntax for `updateOne()`

```
db.collection.updateOne(  
  <filter>,  
  <update>  
)
```

Parameters

- **filter**: Criteria to find the document.
- **update**: Update definition using operators (e.g., `$set`).

Example: Update a Student with Name "Akshay Kumar"

```
db.students.updateOne(
  { name: "Akshay Kumar" },
  {
    $set: {
      age: 15,
      name: "Akshay"
    }
  }
)
```

Outcome:

- Only the first document with name "Akshay Kumar" is updated.

2.2 Using ID instead of Name

IDs are unique; hence preferred for updates.

Example: Update Student's Class Using `_id`

```
db.students.updateOne(
  { _id: ObjectId("64a78f9df32...") },
  {
    $set: { class: "B.Tech" }
  }
)
```

2.3 Syntax for `updateMany()`

```
db.collection.updateMany(
  <filter>,
  <update>
)
```

Example: Update All Students Having Class "BCA"

```
db.students.updateMany(
  { class: "BCA" },
  { $set: { class: "BIT" } }
)
```

Result: All BCA students' class changed to BIT.

3. Update Operators in MongoDB

MongoDB provides various update operators. Below are all used in the transcript with live examples.

3.1 \$set

Updates or adds new fields.

```
db.students.updateOne(
  { name: "Salman" },
  { $set: { age: 24 } }
)
```

3.2 \$unset

Removes a field entirely.

```
db.students.updateOne(
  { name: "Shahid Kapoor" },
  { $unset: { age: "" } }
)
```

3.3 \$rename

Renames a field.

Example: Rename `skills` → `coding_skills` (for all docs)

```
db.students.updateMany(
  {},
  { $rename: { skills: "coding_skills" } }
)
```

3.4 \$inc

Increments a numeric field.

Example: Increase Age of Shahid by 2

```
db.students.updateOne(
  { name: "Shahid Kapoor" },
  { $inc: { age: 2 } }
)
```

3.5 \$mul

Multiplies a numeric field.

Example: Multiply Age by 2

```
db.students.updateOne(
  { name: "Shahid Kapoor" },
  { $mul: { age: 2 } }
)
```

3.6 \$currentDate

Sets field value to current date/time.

Example: Add/Update `lastModified` with current date

```
db.students.updateMany(
  {},
  { $currentDate: { lastModified: true } }
)
```

Result: Adds `lastModified` field to all docs with current timestamp.

3.7 \$min

Replaces field only if the provided value is less than existing value.

Example

```
db.students.updateOne(
  { name: "Salman Khan" },
  { $min: { age: 20 } }
)
```

If existing age = 24 → updated to 20.

3.8 \$max

Opposite of \$min.

```
db.students.updateOne(
  { name: "Salman Khan" },
  { $max: { age: 25 } }
)
```

If provided value > existing, update occurs.

4. Array Update Operators

MongoDB supports special operators for manipulating arrays.

4.1 \$push

Adds a new element to an array.

Example

```
db.students.updateOne(
  { _id: ObjectId("64a7...") },
  { $push: { skills: "Javascript" } }
)
```

4.2 \$addToSet

Adds value only if it does NOT already exist (avoids duplicates).

Example

```
db.students.updateOne(
  { _id: ObjectId("64a7...") },
  { $addToSet: { skills: "Java" } }
)
```

4.3 \$pop

Removes first or last item.

- `1` → remove last
- `-1` → remove first

Example: Remove last element

```
db.students.updateOne(  
  { name: "Salman Khan" },  
  { $pop: { skills: 1 } }  
)
```

4.4 \$pull

Removes specific matching value from array.

Example: Remove "Python"

```
db.students.updateOne(  
  { _id: ObjectId("64a7...") },  
  { $pull: { skills: "Python" } }  
)
```

4.5 \$pullAll

Removes multiple values at once.

Example

```
db.students.updateOne(  
  { _id: ObjectId("64ab...") },  
  { $pullAll: { skills: ["Javascript", "HTML"] } }  
)
```

5. Replacing Documents Using `replaceOne()`

`replaceOne()` replaces the entire document except `_id`.

Syntax

```
db.collection.replaceOne(  
  <filter>,
```

```
<newDocument>  
)
```

Example: Replace Entire Document of Shahid Kapoor

```
db.students.replaceOne(  
  { _id: ObjectId("64a87...") },  
  {  
    name: "Saif Ali Khan",  
    age: 21,  
    skills: ["Go", "C++"]  
  }  
)
```

Outcome:

- Old record removed
- Entirely new record inserted with same `_id`

6. Summary of Covered Topics

Update Commands

- `updateOne()`
- `updateMany()`

Core Update Operators

- `$set`, `$unset`, `$rename`
- `$inc`, `$mul`, `$currentDate`
- `$min`, `$max`

Array Operators

- `$push`, `$addToSet`, `$pop`, `$pull`, `$pullAll`

Replace Command

- `replaceOne()`

The tutorial demonstrates updating single/multiple documents, working with numeric fields, date fields, arrays, and replacing entire documents.
